Post Training Pruning Based on LSA: Local Sensitivity Awareness

Md Ishzaz Asif Rafid
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
ishzaz.rafid@northsouth.edu

Abir Al Mahdi Akhand
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh
abir.akhand@northsouth.edu

***Abstract*—Pruning neural networks has emerged as a crucial technique to reduce model size and computational complexity while preserving accuracy. In this work, we propose a novel pruning strategy called Local Sensitivity Awareness (LSA), which introduces a new perspective in identifying the importance of individual weights within a neural network. Unlike traditional Taylor expansion [1] based methods that consider either gradients or activations independently, our approach leverages the synergy between both weight gradients and their corresponding parent activation gradients. Specifically, LSA computes the importance of each weight as a product of its gradient, absolute value, the parent activation gradient, and the absolute activation. The final importance score is normalized using the sum of absolute parent activation weights to ensure a relative scale across the network. We applied our method to state-of-the-art models like the ResNet architectures [6] including ResNet-50, ResNet-18 and VGG-16 [13], and transformer models like the T5 [12], trained across diverse datasets like TinyImageNet [14], CIFAR-100 [8], CIFAR-10 [8], Facebook AI's PersonaChat [15] Dataset and the Dialogue-Dataset [3] for transformer based models. Extensive experiments demonstrate that pruning up to 50% of weights using LSA preserves competitive accuracy, increases the accuracy in some cases, while significantly reducing model size and computational overhead.**

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Deep neural networks (DNNs) have shown remarkable performance across a wide range of tasks including image classification, object detection, and natural language processing. However, their impressive accuracy often comes at the cost of high computational and memory requirements, hindering deployment on resource-constrained devices. Pruning techniques, which aim to remove redundant parameters from neural networks, have emerged as an effective solution to this challenge.

Traditional pruning approaches rely on magnitude-based criteria, gradient information, or activation sensitivity to determine the importance of weights. While these methods have achieved considerable success, they often neglect the interaction between the weights and the activations within the network. Furthermore, many existing techniques do not take into account the residual nature of weights that may contribute to the stability and generalization ability of the model.

To address these limitations, we introduce a new pruning framework called Local Sensitivity Awareness (LSA). This method synergistically combines gradient-based sensitivity of both weights and their parent activations while preserving the residual contribution of the weights. By normalizing the importance score using the sum of parent activation weights, LSA ensures a relative and balanced importance distribution across layers. Our contributions are summarized as follows:

1) We propose a new importance scoring mechanism based on the combined Taylor scores of weights and activations.
2) We introduce a relative normalization scheme to fairly compare weights within each layer.
3) We evaluate the effectiveness of LSA through extensive experiments on several benchmark datasets, including CIFAR-100 [8], CIFAR-10 [8], and TinyImageNet [14], utilizing deep learning models such as ResNet and Transformer architectures.

The results demonstrate that LSA is able to reduce model complexity significantly without sacrificing, and in some cases even improving, predictive accuracy. Our findings highlight the potential of LSA as a powerful pruning method for optimizing neural networks, especially in scenarios where computational resources are limited, and performance is paramount.

II. Related Works

Post-training pruning (PTP) has become a practical method for model compression, particularly for large models where retraining is costly. Zhang et al. (2023) [16] introduce a plug-and-play pruning method for large language models (LLMs) that combines Relative Importance and Activations (RIA) with Channel Permutation [11] to achieve hardware-friendly N:M sparsity. RIA

mitigates channel corruption by considering both weight magnitudes and activation dynamics, achieving up to 50% sparsity with minimal performance loss, sometimes outperforming dense models in zero-shot tasks. This method, which incorporates activations into the pruning criteria, echoes the motivation behind our approach. However, unlike Zhang et al., which primarily focuses on magnitude-based metrics, our approach (LSA) utilizes a more refined formula that combines weight magnitudes, gradients, and activations, providing a more nuanced importance measure. Cheng et al. (2024) [2], in their comprehensive survey "A Survey on Deep Neural Network Pruning", categorized pruning techniques based on when and how pruning is performed, such as before, during, or after training, and whether pruning is structured or unstructured. Their work highlighted the increasing importance of considering both weight sensitivity and activation behavior while designing pruning strategies. However, they also noted a gap in methods that incorporate residual characteristics of weights, especially in architectures like ResNet. Zhao et al. [17] address the challenge of deploying large language models (LLMs) with massive parameters and computations. They focus on post-training pruning methods, which prune LLMs in one-shot without the expensive retraining process typically required by traditional pruning techniques. The paper argues that existing post-training pruning methods, such as SparseGPT [4], are sub-optimal because they rely on solving the Single Removal Problem (SRP) to address the pruning of multiple weights, which is inherently a Multiple Removal Problem (MRP). To overcome this, they formulate the MRP for layer-wise LLM pruning and derive an optimal solution, demonstrating superior performance compared to state-of-the-art baselines. Our method is related as it also addresses the problem of pruning, particularly in a post-training setting, but it uses a different approach to determine weight importance and doesn't specifically focus on the Single vs. Multiple Removal Problem, rather focuses on the incorporation of activation gradients along with the weight gradients normalizing it with the summation of the parent activation weights. Gaikwad and El-Sharkawy [5] explore pruning convolutional neural networks (CNNs) using Taylor expansion [1]. They propose a method to prune CNNs based on the Taylor expansion of the change in the cost function. Their algorithm employs a greedy criterion for pruning and fine-tunes the network using backpropagation, demonstrating its effectiveness on the SqueezeNet [7] architecture with the CIFAR-10 [8] dataset. Transfer learning [10] is utilized to train the SqueezeNet model. This work is similar to ours in the use of Taylor expansion to guide pruning, but it differs in the network architecture (CNN vs. ResNet/VGG), the specific Taylor-based criterion, and the fine-tuning approach. Also, our method calculates the taylor scores of the weight itself and its parent activation. Lazarevich et al. [9] present a post-training weight pruning method for deep neural networks that aims to achieve high accuracy while being computationally efficient. They also propose a data-free extension of the approach for computer vision models based on automatically-generated synthetic fractal images. We obtain state-of-the-art results for data-free neural network pruning, with 1.5% top@1 accuracy drop for a ResNet50 on ImageNet at 50% sparsity rate. Their method achieves state-of-the-art results in data-free neural network pruning and demonstrates strong performance when using real data, even at high sparsity rates. However, our method uses a unique weight importance formula based on Taylor scores and normalization, rather than the techniques used in this paper, and achieves better results than this paper. Motivated by these works, our proposed Local Sensitivity Awareness (LSA) method combines the Taylor scores of weights and their parent activations while normalizing it with the sum of the absolute values of the weights of the parent activation, addressing the gaps in existing pruning methods and improving the robustness of pruning across diverse architectures and datasets.

III. Methodology

In this section, we provide a comprehensive explanation of our proposed pruning framework, Local Sensitivity Awareness (LSA). The goal of LSA is to effectively identify and prune the least important weights of a neural network while preserving model accuracy even at high pruning ratios. Unlike traditional pruning techniques that rely on either weight magnitude or gradient sensitivity alone, LSA introduces a novel importance score that incorporates the sensitivity of both weights and activations, while also respecting the residual contribution of the weights themselves. Additionally, the method normalizes the importance scores based on the local context of the weights within their parent activation, ensuring a balanced and relative importance evaluation across different layers and neurons.

A. Importance Calculation

At the core of our pruning method is the calculation of weight and activation importance. In order to calculate the importance of each weight, we propose the following formula:

$$I = \frac{\left(\frac{\partial L}{\partial w} \times |w| \times \frac{\partial L}{\partial A} \times |A|\right)}{\sum_{i=1}^{n} |w_i|} \tag{1}$$

where w is the weight parameter, A represents the activation of the neuron, $\frac{\partial L}{\partial w}$ is the gradient of the loss function L with respect to the weight w, $\frac{\partial L}{\partial A}$ is the gradient of the loss function with respect to the

activation A, $|A|$ is the absolute value of the activation, and $\sum_{i=1}^{n} |w_i|$ serves to normalize the importance measure across all weights.

Our proposed equation combines several key components to calculate the importance of each weight in the network. First, it accounts for the absolute magnitude of the weight $|w|$, which reflects the weight's contribution to the network's function. Second, it incorporates the gradient of the loss function with respect to both the weight ($\frac{\partial L}{\partial w}$) and the activation ($\frac{\partial L}{\partial A}$). These gradients capture how sensitive the model's performance is to changes in the weight and activation, respectively. By including the absolute value of the activation $|A|$ and weight $|w|$, we consider the contribution of each activation and weight to the output of the model. The sum of absolute weights in the denominator normalizes the importance measure, ensuring consistency across different layers and architectures.

This formulation brings together several critical insights. If both the weight and its activation are highly sensitive to loss, the product of gradients amplifies their importance. Larger weights and activations signify greater influence during forward and backward propagation, which is captured via their magnitudes. Even if the gradients are small due to optimization convergence. Normalizing by the sum of parent weights prevents a bias towards layers with inherently larger weights, ensuring fair relative scoring.

B. Pruning Procedure

The pruning process using the LSA method involves several steps, which are described below with appropriate diagram:

1) Model Initialization: The neural network is first trained on a given dataset (e.g., CIFAR-10 [8], CIFAR-100 [8], or TinyImageNet [14]). This training phase ensures that the model learns the necessary features for accurate predictions, and the trained model serves as the base for pruning.

2) Importance Calculation: After training, the importance of each weight is calculated using the LSA formula. This step involves computing the gradients ($\frac{\partial L}{\partial w}$) and ($\frac{\partial L}{\partial A}$), as well as evaluating the activations A. The importance score for each weight is derived from these components, which encapsulate the weight's contribution to both the loss and the activation dynamics of the network.

3) Weight Selection: Once the importance values are calculated, a threshold is set to identify weights that have relatively low importance. These weights are considered less crucial for the model's predictive power and are selected for pruning. The importance values allow us to rank the weights and prune the least important ones.

4) Pruning and Model Compression: The identified weights are removed from the network, resulting in a compressed model with fewer parameters. This reduces the computational and memory requirements of the model, making it more suitable for deployment in resource-constrained environments such as mobile devices or edge computing platforms.

5) Evaluation: After pruning, the model is evaluated on the test dataset to assess its performance. The accuracy of the pruned model is compared to that of the original model to determine how well pruning has preserved the model's performance. In some cases, pruning may lead to improved performance due to the removal of redundant or non-contributory parameters.

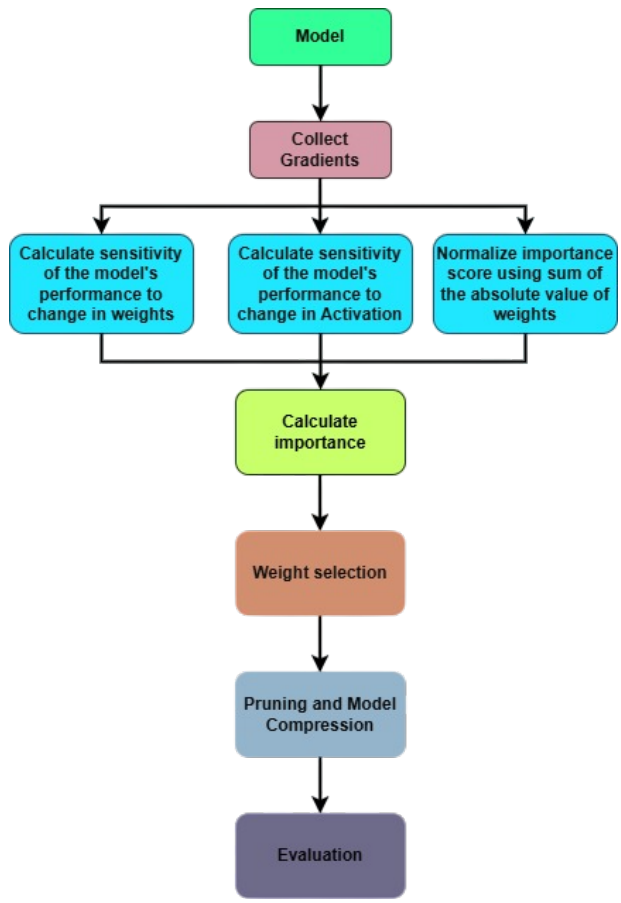

Fig. 1. Methodology Flowchart

6) Application on different Architectures and Datasets: We evaluate the effectiveness of the LSA pruning method across multiple popular neural network architectures and diverse datasets to validate its general applicability and robustness. The Architectures we

have used are ResNet-50 [6], ResNet-18 [6], VGG-16 [13], T5 [12] and the Datasets used are CIFAR-10 [8], CIFAR-100 [8], TinyImageNet [14], Dialogue-Dataset [3] and PersonaChat [15]. We have trained the models with the mentioned datasets in 20 epochs with NVidia GeForce Rtx4070 with DDR5 32GB and an Intel Core i5 13400f Processor. This diverse experimental setup ensures that LSA is not limited to any particular network type or dataset and can generalize well across both vision-based convolutional networks and transformer-based language models. Experimental results are shown in Table I.

C. Comparison with Traditional Methods

As previously stated, we applied our algorithm to various architectures trained on different datasets. We then compared our results with several well-known pruning algorithms, and the results are shown a visual representation in Image 2.

Traditional pruning methods, such as magnitude pruning and first-order Taylor [1] expansion-based pruning, primarily rely on estimating the importance of parameters based on the magnitude of the weights or the first derivative of the loss function. These methods often fail to capture the complex interactions between weights, gradients, and activations, leading to suboptimal pruning decisions. For instance, magnitude pruning considers only the size of the weight, while first-order Taylor expansion-based methods focus on the local linear behavior of the loss function with respect to the weights, the influence of the activations that can significantly affect model performance.

In contrast, our LSA method integrates weight gradients and activations into a single unified framework for calculating weight importance. By considering both the magnitude of the weight and its relationship with the activation and loss gradients, LSA provides a more comprehensive evaluation of each parameter's significance. This enables more accurate pruning decisions that better preserve the overall model performance, as demonstrated in our experimental results.

D. Experimental Setup

To validate the effectiveness of LSA, we perform experiments on widely used benchmark datasets, including CIFAR-10 [8], CIFAR-100 [8], and TinyImageNet [14]. We test the LSA pruning method on various model architectures, including ResNets [6] and Transformer-based models, to evaluate its generalizability across different network types. The performance of pruned models is compared with that of the original, unpruned models in terms of accuracy as shown in Table I. The comparative analysis of of different pruning methods in different architectures and datasets at 10% pruning ratio is visualized in Image 3.

E. Advantages of LSA

The LSA method provides several key advantages over traditional pruning techniques:

1) Higher Accuracy Preservation and Increase in Accuracy: By considering higher-order terms, gradients, and activations, LSA is able to more accurately assess the importance of each parameter. This leads to more effective pruning decisions, ensuring that crucial weights are retained and model accuracy is preserved. Also, experiments have showed that models like the ResNet [6] Architectures often tend to **improve the accuracy** in lower pruning rate.

2) Improved Model Compression: LSA achieves significant model compression, reducing the number of parameters with comparatively less sacrifice of accuracy. This is particularly beneficial for deploying models on edge devices with limited computational and memory resources. For example, then accuracy drop of RESNET50 [6] trained on TinyImageNet [14] dataset is shown in fig-4.

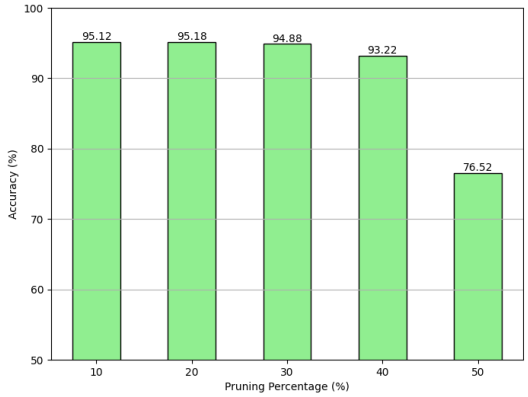

Fig. 4. Accuracy drop of RESNET50 trained on TinyImageNet Dataset

Here, the accuracy of the mentioned model remains consistently around 93%-96% till 40% pruning. But at 50%, the accuracy drops to 76.52%, proving the effective consistency of our approach in maintaining accuracy till 40% pruning rate. However, in case of LLM, our model has chances to perform better. The T5 LLM trained on the Dialogue [3] and PersonaChat [15] datasets sees a massive accuracy drop throughout the pruning rates, demonstrated by the graph in fig 5

TABLE I
PERFORMANCE OF LSA ON VARIOUS MODELS AND DATASETS

Model	Dataset	Accuracy(%)	Post Pruning Accuracy (%)					Accuracy Difference				
			10%	20%	30%	40%	50%	10%	20%	30%	40%	50%
RESNET50 [6]	CIFAR10 [8]	84.375	90.62	84.38	84.38	84.38	90.62	+6.245	0.005	0.005	0.005	+6.245
	CIFAR100 [8]	87.5	87.50	87.50	87.50	87.50	84.38	0.00	0.00	0.00	0.00	3.12
	Tiny ImageNet [14]	95.72	95.12	95.18	94.88	93.22	76.52	0.60	0.54	0.84	2.50	19.20
RESNET18 [6]	CIFAR10 [8]	100.0	100.00	100.00	100.00	100.00	96.88	0.00	0.00	0.00	0.00	3.12
	CIFAR100 [8]	81.25	84.38	81.25	78.12	78.12	81.25	+3.13	0.00	3.13	3.13	0.00
	Tiny ImageNet [14]	90.68	90.70	89.36	89.02	84.68	65.72	+0.02	1.32	1.66	6.00	24.96
VGG16 [13]	CIFAR10 [8]	100.0	100.00	100.00	96.88	93.75	81.25	0.00	0.00	3.12	6.25	18.75
	CIFAR100 [8]	78.125	78.12	78.12	78.12	75.00	34.38	0.005	0.005	0.005	3.125	43.745
	Tiny ImageNet [14]	59.66	59.22	59.58	58.48	52.46	9.14	0.44	0.08	1.18	7.20	50.52
T5 [12]	DialogueDataset [3]	98.46	98.46	79.46	70.45	43.92	27.28	0.00	19.00	28.01	54.54	71.18
	Personachat [15]	92.466	89.734	76.216	68.099	44.728	27.165	2.732	16.25	22.016	48.546	65.186

The Positive values under the column "Accuracy Difference" denotes an **Increase in Accuracy** after pruning.

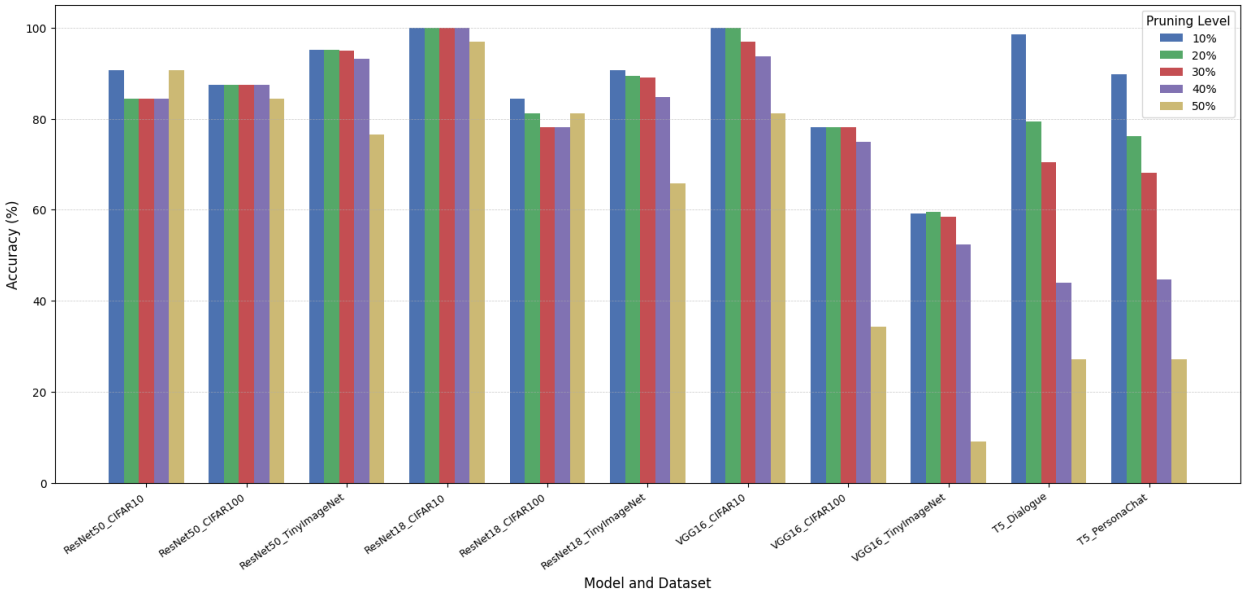

Fig. 2. Accuracy of our algorithm across different pruning percentages in different models

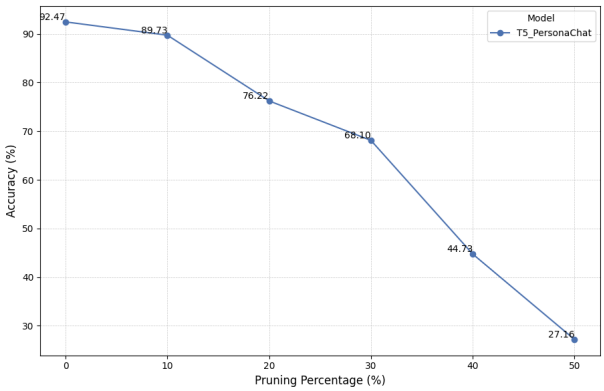

Fig. 5. Accuracy drop of T5 trained on PersonaChat [15] Dataset

Here, the accuracy of the T5 transformer is 98.46% after 10% pruning, which is good. However, the accuracy starts to drop from 20% pruning, which eventually reaches to 27.28% at 50% pruning rate.

3) Reduced Dependency on Retraining: As a post-training pruning method, LSA does not require retraining or fine-tuning after pruning. This eliminates the need for additional computational resources and makes the pruning process more efficient.

4) Comprehensive Evaluation of Parameter Importance: Unlike traditional pruning methods that rely on simplistic criteria, LSA incorporates a holistic measure of weight importance by considering multiple factors, including activations and gradients. This leads to a more nuanced and accurate pruning process.

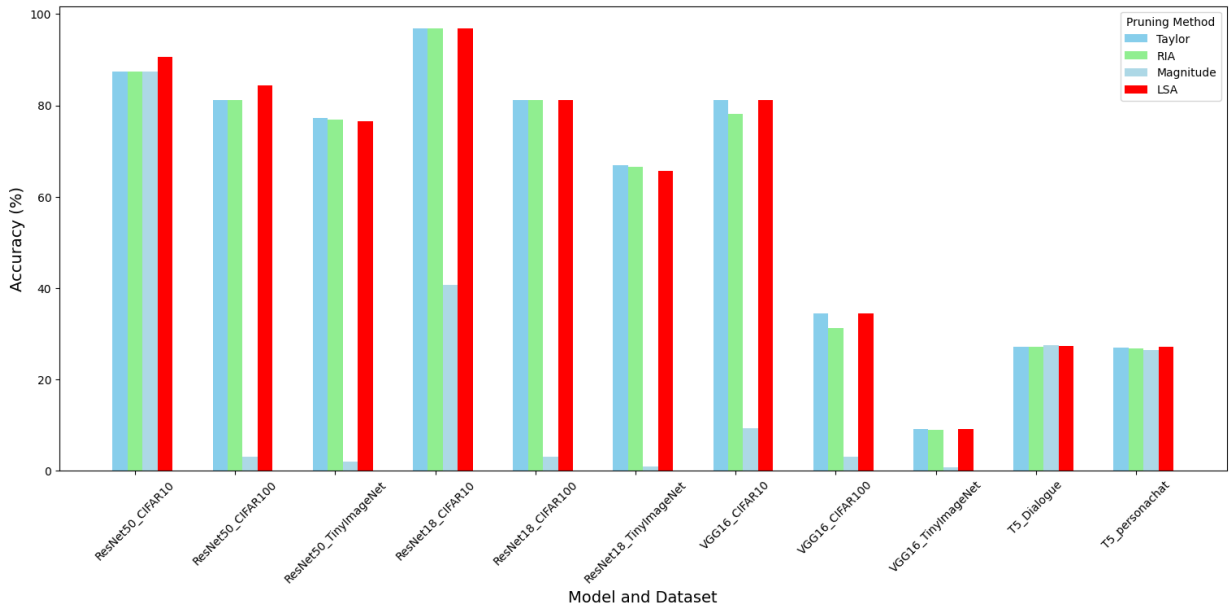

Fig. 3. Comparison of Pruning methods at 10% pruning

5) Increase in Accuracy at lower pruning rates: Our pruning method demonstrates significant performance improvements in ResNet50 and ResNet18 models when applied to the CIFAR10 and CIFAR100 datasets, respectively. Specifically, ResNet50 trained on CIFAR10 achieves a 6.25% accuracy increase at 10% pruning, while ResNet18 trained on CIFAR100 exhibits a 3.13% improvement under the same pruning ratio. The graphs in fig 6 and fig 7 below illustrate these results in detail.

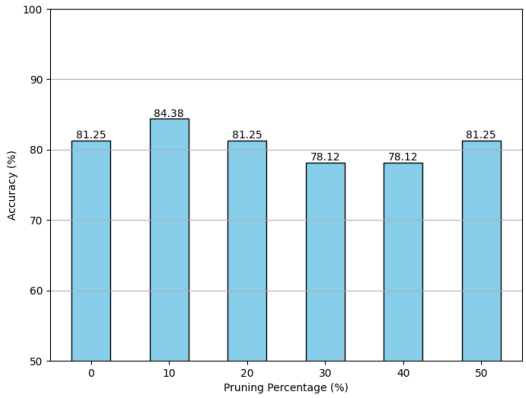

Fig. 6. Increase in accuracy at 10% pruning, Resnet18 trained on CIFAR100

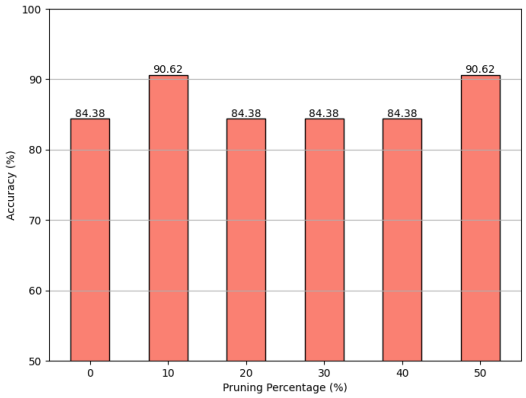

Fig. 7. Increase in accuracy at 10% and 50% pruning, Resnet50 trained on CIFAR100

Moreover, it avoids the risk of pruning critical weights solely based on low gradient values. It is architecture-agnostic and works on both vision models and language models. It performs well even at high pruning ratios (up to 50%) without significant accuracy drop, rather a slight accuracy increase in some cases. The method is simple, interpretable, and does not require complex additional training or hyperparameter tuning.

IV. Conclusion

In this work, we have introduced a novel and effective neural network pruning framework named Local Sensi-

tivity Awareness (LSA), which addresses several critical limitations present in existing pruning strategies. As deep learning models continue to grow in size and complexity, their deployment in real-world, resource-constrained environments such as mobile devices, embedded systems, and edge computing platforms becomes increasingly difficult. Pruning, as one of the most important model compression techniques, plays a vital role in reducing computational costs, memory footprint, and energy consumption while maintaining competitive accuracy.

The primary contribution of our work lies in the design of a new importance scoring method for weight pruning, which simultaneously considers multiple key factors that contribute to the true significance of a weight in a neural network. Specifically, unlike traditional methods that rely solely on weight magnitude or sensitivity with respect to the loss function, LSA goes beyond by introducing a more holistic scoring mechanism. Our method incorporates both the Taylor expansion-based gradients of the weight and its parent activation, the absolute magnitudes of the weight and activation.

Moreover, LSA normalizes the calculated importance score using the sum of the absolute values of the weights within the same parent activation group. This relative normalization allows the pruning strategy to fairly compare the importance of weights across different layers and neurons, regardless of their inherent scale differences. As a result, LSA achieves a balance between sensitivity-driven pruning and magnitude-based pruning, making it suitable for both structured and unstructured pruning scenarios.

Another significant advantage of our approach is its flexibility and generalizability. The proposed LSA method was successfully applied to a wide range of popular neural network architectures, including ResNet-50 [6], ResNet-18 [6], VGG-16 [13], and T5 [12]. Furthermore, we evaluated LSA across multiple datasets covering various tasks and domains, including CIFAR-10 [8], CIFAR-100 [8], TinyImageNet [14], PersonaChat [15] and the Dialogue datasets [3]. The consistent performance of LSA across these diverse models and datasets highlights its robustness and adaptability.

Our extensive experimental results demonstrated that LSA is capable of pruning a significant portion of the network's parameters—up to 50%—while maintaining competitive accuracy and minimal performance degradation. In several cases, the accuracy drop was negligible, indicating the effectiveness of our importance scoring mechanism in identifying truly unimportant weights without harming the network's representational power. In some cases, a slight accuracy increase is also seen. This property is particularly valuable for real-world scenarios where aggressive pruning is necessary to meet strict latency, memory, or power constraints.

Additionally, the simplicity of our method is another strength worth noting. LSA does not require complicated additional modules, costly iterative retraining processes, or hyperparameter tuning to perform well. The importance score is computed in a straightforward manner based on readily available gradient and weight information during backpropagation. This makes LSA easy to implement and integrate into existing neural network training or compression pipelines.

In conclusion, LSA presents a powerful, general, and easy-to-implement solution to neural network pruning that effectively captures the combined influence of weight gradients, activation gradients and the weight magnitude. Its architecture-agnostic design, excellent performance across multiple datasets, and robustness under high pruning ratios establish LSA as a promising approach for practical model compression.

In future work, we aim to explore further enhancements to LSA by extending it to dynamic pruning scenarios, where the pruning decisions can adapt during training based on evolving importance scores. Additionally, integrating LSA with other model compression techniques, such as quantization or knowledge distillation, could lead to even more compact and efficient deep learning models suitable for deployment in highly resource-constrained environments like IoT devices and autonomous systems.

REFERENCES

[1] Mohsen Pourahmadi and. Taylor expansion of and some applications. *The American Mathematical Monthly*, 91(5):303–307, 1984.

[2] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):10558–10578, 2024.

[3] endofnight17j03. Dialogue dataset. https://www.kaggle.com/datasets/endofnight17j03/dialogue-dataset, 2023. Accessed: 2025-04-13.

[4] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10323–10337. PMLR, 23–29 Jul 2023.

[5] Akash Sunil Gaikwad and Mohamed El-Sharkawy. Pruning convolution neural network (squeezenet) using taylor expansion-based criterion. In *2018 IEEE International Symposium on Signal Processing and Information Technology (ISSPIT)*, pages 1–5. IEEE, 2018.

[6] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

[7] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and ¡0.5mb model size, 2016.

[8] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[9] Ivan Lazarevich, Alexander Kozlov, and Nikita Malinin. Post-training deep neural network pruning via layer-wise calibration, 2021.

[10] Sinno Jialin Pan. Transfer learning. *Learning*, 21:1–2, 2020.
[11] Jeff Pool and Chong Yu. Channel permutations for n:m sparsity. In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 13316–13327. Curran Associates, Inc., 2021.
[12] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.
[13] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2015.
[14] Xuan Yang. Tiny imagenet visual recognition challenge.
[15] Saizheng Zhang, Emily Dinan, Jack Urbanek, Arthur Szlam, Douwe Kiela, and Jason Weston. Personalizing dialogue agents: I have a dog, do you have pets too?, 2018.
[16] Yingtao Zhang, Haoli Bai, Haokun Lin, Jialin Zhao, Lu Hou, and Carlo Vittorio Cannistraci. Plug-and-play: An efficient post-training pruning method for large language models. In *The Twelfth International Conference on Learning Representations*, 2024.
[17] Pu Zhao, Fei Sun, Xuan Shen, Pinrui Yu, Zhenglun Kong, Yanzhi Wang, and Xue Lin. Pruning foundation models for high accuracy without retraining, 2024.